

자바의 기본 입출력

자바의 기본 출력

Module java.base

Package java.lang

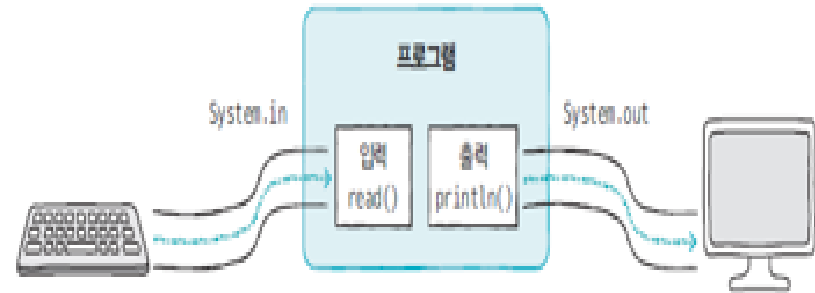
Class System

java.lang.Object

java.lang.System

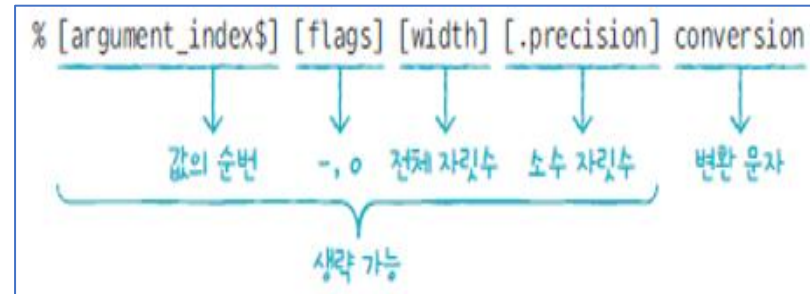
```
public final class System  
extends Object
```

- System.out.println 구문
- System
 - 표준 입력(키보드), 표준 출력(모니터) 및 오류 출력 기능 제공
 - **public static final InputStream in**
 - **public static final PrintStream out**
 - **public static final PrintStream err**
- System.out
 - 출력 스트림에 다양한 데이터 값을 편리하게 출력하는 기능
- System.out.println()
 - 출력할 인자와 개행 정보를 표준 콘솔 장치에 전달하는 함수



자바의 기본 출력

- System.out.print()
 - 출력하고 줄바꿈 하지 않음
- System.out.printf("형식문자열", 값1, 값2,...)



형식화된 문자열		설명	출력 형태
정수	%d	정수	123
	%6d	6자리 정수. 왼쪽 빈 자리 공백	___123
	%-6d	6자리 정수. 오른쪽 빈 자리 공백	123___
	%06d	6자리 정수. 왼쪽 빈 자리 0 채움	000123
실수	%10.2f	소수점 이상 7자리, 소수점 이하 2자리. 왼쪽 빈 자리 공백	___123.45
	%-10.2f	소수점 이상 7자리, 소수점 이하 2자리. 오른쪽 빈 자리 공백	123.45___
	%010.2f	소수점 이상 7자리, 소수점 이하 2자리. 왼쪽 빈 자리 0 채움	0000123.45
문자열	%s	문자열	abc
	%6s	6자리 문자열. 왼쪽 빈 자리 공백	___abc
	%-6s	6자리 문자열. 오른쪽 빈 자리 공백	abc___
특수 문자	\t	탭(tab)	
	\n	줄 바꿈	
	%%	%	%

System.out.printf("이름: %1\$s, 나이: %2\$d", "김자바", 25); → 이름: 김자바, 나이: 25

예제. printf를 이용한 형식 출력

- printf 활용
 - 서식 지정자로 자릿수와 정렬을 맞춰 표 형태로 출력할 수 있음
 - %5d : 5자리 폭 오른쪽 정렬, %5.1f : 전체 5자리·소수 1자리, %n : 줄바꿈

*운영체제에 따라 다름
-. Window : %r%n
-. Linux/Mac : %n

<코드>

```
String name = "홍길동";  
int kor = 95, eng = 88;  
double avg = (kor + eng) / 2.0;  
  
System.out.printf("이름 : %s%n", name);  
System.out.printf("국어 : %5d점%n", kor);  
System.out.printf("영어 : %5d점%n", eng);  
System.out.printf("평균 : %5.1f점%n", avg);
```

<출력 예시>

```
이름 : 홍길동  
국어 :    95점  
영어 :    88점  
평균 :   91.5점
```

자바의 키 입력과 System.in

- System.in
 - 키보드와 연결된 자바의 표준 입력 스트림
 - 입력되는 키를 바이트(문자 아님)로 리턴하는 저수준 스트림
 - 입력 버퍼에 저장되어 Backspace키를 이용한 편집도 가능
 - Enter키를 누르면 입력된 데이터를 읽어냄
 - System.in을 직접 사용하면 **바이트를 문자나 숫자로 변환하는 많은 어려움 있음**

읽은 키코드를 변수에 저장

```
int keyCode = System.in.read();
```

시스템이 가지고 있는 입력 장치에서 입력된 키코드를 읽어라

숫자	알파벳				기능키	방향키
0 = 48	A = 65	N = 78	a = 97	n = 110	BACK SPACE = 8	← = 37
1 = 49	B = 66	O = 79	b = 98	o = 111	TAB = 9	↑ = 38
2 = 50	C = 67	P = 80	c = 99	p = 112	ENTER = [CR=13, LF=10]	→ = 39
3 = 51	D = 68	Q = 81	d = 100	q = 113	SHIFT = 16	↓ = 40
4 = 52	E = 69	R = 82	e = 101	r = 114	CONTROL = 17	
5 = 53	F = 70	S = 83	f = 102	s = 115	ALT = 18	
6 = 54	G = 71	T = 84	g = 103	t = 116	ESC = 27	
7 = 55	H = 72	U = 85	h = 104	u = 117	SPACE = 32	
8 = 56	I = 73	V = 86	i = 105	v = 118	PAGEUP = 33	
9 = 57	J = 74	W = 87	j = 106	w = 119	PAGEDN = 34	
	K = 75	X = 88	k = 107	x = 120		
	L = 76	Y = 89	l = 108	y = 121		
	M = 77	Z = 90	m = 109	z = 122		

자바의 키 입력과 System.in

- System.in.read()
 - end of stream (-1)을 만날 때까지 동작
 - Ctrl+z+Enter(윈도우) 또는 Ctrl+d(Linux/Mac)
 - IntelliJ 콘솔에서는 Ctrl+d
 - **입력 스트림이 종료되면 키보드 입력 받을 수 없음**
 - IOException 예외처리 구문 포함해야 사용 가능
`public static void main(String[] args) throws IOException`

```
int keyCode = System.in.read();
System.out.println("keyCode : "+keyCode);

keyCode = System.in.read();
System.out.println("keyCode : "+keyCode);

keyCode = System.in.read();
System.out.println("keyCode : "+keyCode);
```

a

```
keyCode : 97
keyCode : 13
keyCode : 10
```

abc

```
keyCode : 97
keyCode : 98
keyCode : 99
```

a b c

```
keyCode : 97
keyCode : 32
keyCode : 98
```

*운영체제에 따라 다름
-. Window : \r\n
-. Linux/Mac : \n

실습 예제. 키보드 입력값 확인하기

- 실습내용
 - 키를 누르고 엔터를 치면 키값과 문자 출력하기를 반복
 - -1이 입력될 때까지 무한 반복
 - 출력 예시

a

입력문자 : a, code : 97

입력문자 :

, code : 10

b

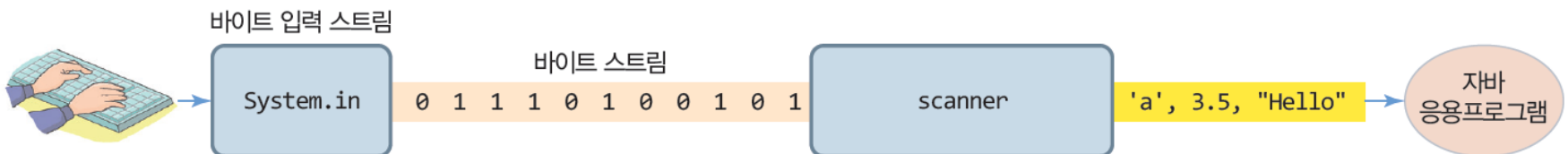
입력문자 : b, code : 98

입력문자 :

, code : 10

Scanner와 Scanner 객체 생성

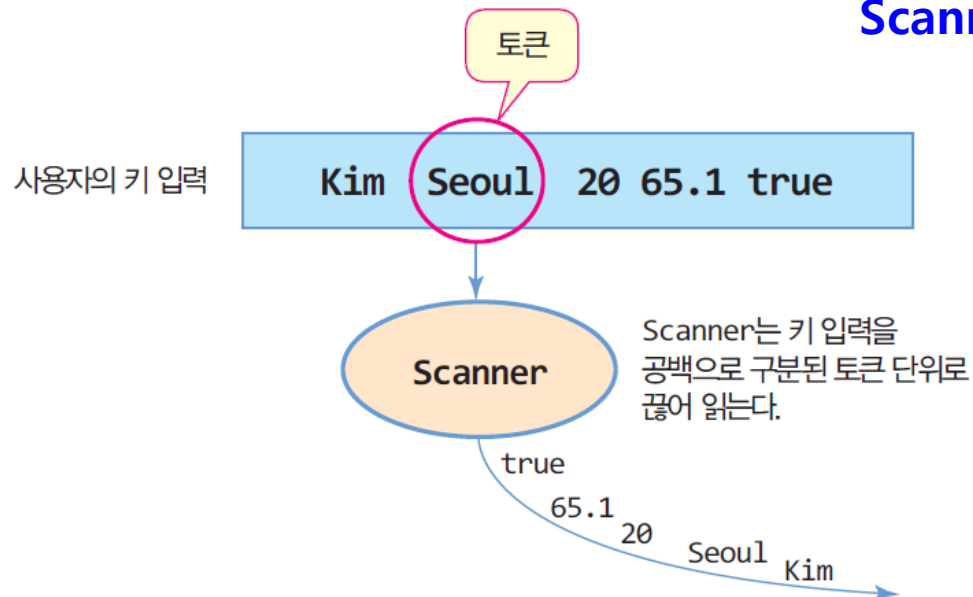
- Scanner 클래스
 - 읽은 바이트를 문자, 정수, 실수, 불린, 문자열 등 다양한 타입으로 변환하여 리턴
 - java.base 모듈내의 java.util 패키지에 정의 되어 있음
 - 객체 생성
- ```
import java.util.Scanner; // 임포트 문 필요
...
Scanner a = new Scanner(System.in); // Scanner 객체 생성
```
- 키보드에 연결된 System.in에게 키를 읽게 하고, 원하는 타입으로 변환하여 리턴



# Scanner를 이용한 키 입력

- Scanner에서 키 입력 받기
  - Scanner는 입력되는 키 값을 공백으로 구분되는 토큰 단위로 읽음
  - 공백 문자 : ' ', '\t', '\f', '\r', '\n'
- 개발자가 원하는 타입 값으로 쉽게 읽을 수 있음

**Scanner scanner = new Scanner(System.in);**



```
String name = scanner.next(); // "Kim"
String city = scanner.next(); // "Seoul"
int age = scanner.nextInt(); // 20
double weight = scanner.nextDouble(); // 65.1
boolean single = scanner.nextBoolean(); // true
```

# Scanner 주요 메소드

| 메소드                              | 설명                                                                                                           |
|----------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>String next()</code>       | 다음 토큰을 문자열로 리턴                                                                                               |
| <code>byte nextByte()</code>     | 다음 토큰을 byte 타입으로 리턴                                                                                          |
| <code>short nextShort()</code>   | 다음 토큰을 short 타입으로 리턴                                                                                         |
| <code>int nextInt()</code>       | 다음 토큰을 int 타입으로 리턴                                                                                           |
| <code>long nextLong()</code>     | 다음 토큰을 long 타입으로 리턴                                                                                          |
| <code>float nextFloat()</code>   | 다음 토큰을 float 타입으로 리턴                                                                                         |
| <code>double nextDouble()</code> | 다음 토큰을 double 타입으로 리턴                                                                                        |
| <code>String nextLine()</code>   | '\n'을 포함하는 한 라인을 읽고 '\n'을 버린 나머지만 리턴                                                                         |
| <code>void close()</code>        | Scanner의 사용 종료                                                                                               |
| <code>boolean hasNext()</code>   | 현재 입력된 토큰이 있으면 true, 아니면 새로운 입력이 들어올 때까지 무한정 기다려서, 새로운 입력이 들어오면 그 때 true 리턴. ctrl-z 키가 입력되면 입력 끝이므로 false 리턴 |

# 실습 예제. Scanner를 이용한 키 입력 연습

- 문제 설명

- Scanner를 이용하여 학번, 이름(공백포함)을 엔터로 구분하여 입력 받고, 입력 받은 내용을 출력하는 프로그램을 작성하라.

학번 : 20000

이름 : Hong GilDong

당신의 이름은 Hong GilDong이고, 학번은 20000입니다.

- 입력 버퍼에 남아있는 엔터로 인한 문제 확인 및 해결
  - `nextLine()`을 이용한 개행 문자 제거

**연산자**

# 연산자

- 연산(operation)
  - 주어진 식을 계산하여 결과를 얻어내는 과정

| 연산의 종류        |     | 연산자                                 |
|---------------|-----|-------------------------------------|
| 우선<br>순위<br>↓ | 증감  | ++ --                               |
|               | 산술  | + - * / %                           |
|               | 시프트 | >> << >>>                           |
|               | 비교  | > < >= <= == !=                     |
|               | 비트  | &   ^ ~                             |
|               | 논리  | &&    ! ^                           |
|               | 조건  | ? :                                 |
|               | 대입  | = *= /= += -= &= ^=  = <<= >>= >>>= |

식(expression)

a + 5

n > 23

a == n

피연산자(operand)

연산자(operator)

피연산자(operand)

# 조건 연산자 ?:

- opr1?opr2:opr3
  - 3 개의 피연산자로 구성된 삼항(ternary) 연산자
    - opr1이 true이면, 연산식의 결과는 opr2, false이면 opr3
  - if-else를 조건연산자로 간결하게 표현 가능

x와 y 중에서  
큰 값을 big에 저장

```
int x = 5;
int y = 3;
```

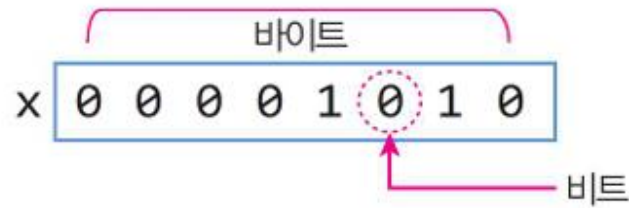
```
int big;
if(x>y)
 big = x;
else
 big = y;
```

```
int big = (x>y)?x:y;
```

# 비트 연산자

- 비트 개념

byte x = 10;



- 1 byte = 8 bit
- 비트 연산
  - 논리 연산
    - 비트끼리 AND, OR, XOR, NOT 연산
  - 시프트 연산
    - 비트를 오른쪽이나 왼쪽으로 이동

# 비트 논리 연산자

- 피 연산자의 각 비트들을 대상으로 하는 연산

$$\begin{array}{r} 01101010 \\ \& 11001101 \\ \hline 01001000 \end{array}$$

모두 1이므로  
결과는 1

둘 중 하나라도  
0이 되면 결과는 0

$$\begin{array}{r} 01101010 \\ | 11001101 \\ \hline 11101111 \end{array}$$

모두 0이므로  
결과는 0

둘 중 하나라도  
1이 되면 결과는 1

$$\begin{array}{r} 01101010 \\ ^ 11001101 \\ \hline 10100111 \end{array}$$

두 비트가 같으므로  
결과는 0

두 비트가 다르므로  
결과는 1

$$\begin{array}{r} \sim 01101010 \\ \hline 10010101 \end{array}$$

1은 0으로 변환

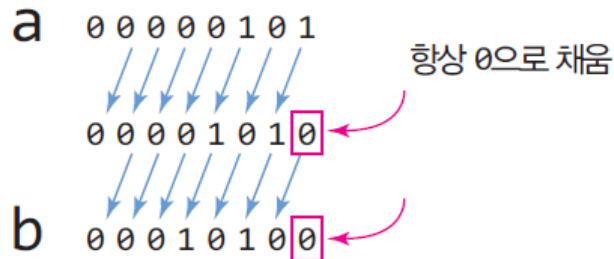
0은 1로 변환

| 비트 연산자   | 내용                                                   |
|----------|------------------------------------------------------|
| $a \& b$ | a와 b의 각 비트들의 AND 연산. 두 비트 모두 1일 때만 1이 되며 나머지는 0이 된다. |
| $a   b$  | a와 b의 각 비트들의 OR 연산. 두 비트 모두 0일 때만 0이 되며 나머지는 1이 된다.  |
| $a ^ b$  | a와 b의 각 비트들의 XOR 연산. 두 비트가 서로 다르면 1, 같으면 0이다.        |
| $\sim a$ | 단항 연산자로서 a의 각 비트들에 NOT 연산. 1을 0으로, 0을 1로 변환한다.       |

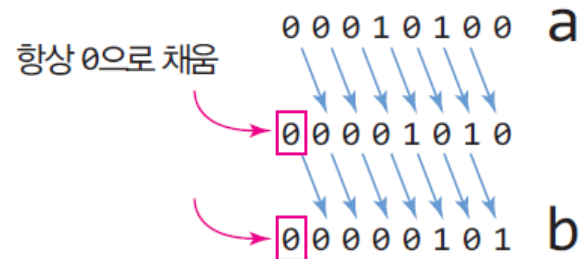
# 시프트 연산자

| 시프트 연산자    | 내용                                                                             |
|------------|--------------------------------------------------------------------------------|
| $a \gg b$  | a의 각 비트를 오른쪽으로 b번 시프트한다. 최상위 비트의 빈자리는 시프트 전의 최상위 비트로 다시 채운다. 산술적 오른쪽 시프트라고 한다. |
| $a \ggg b$ | a의 각 비트를 오른쪽으로 b번 시프트한다. 그리고 최상위 비트의 빈자리는 0으로 채운다. 논리적 오른쪽 시프트라고 한다.           |
| $a \ll b$  | a의 각 비트를 왼쪽으로 b번 시프트한다. 그리고 최하위 비트의 빈자리는 0으로 채운다. 산술적 왼쪽 시프트라고 한다.             |

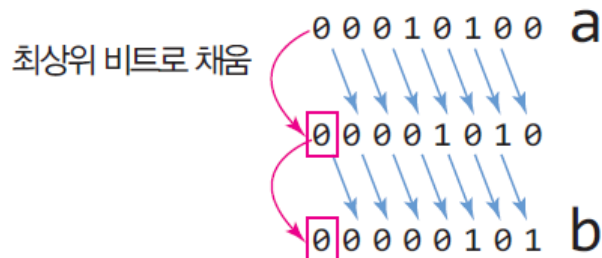
```
byte a = 5; // 5
byte b = (byte)(a << 2); // 20
```



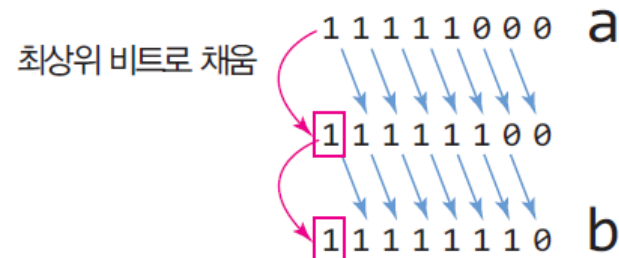
```
byte a = 20; // 20
byte b = (byte)(a >>> 2); // 5
```



```
byte a = 20; // 20
byte b = (byte)(a >> 2); // 5
```

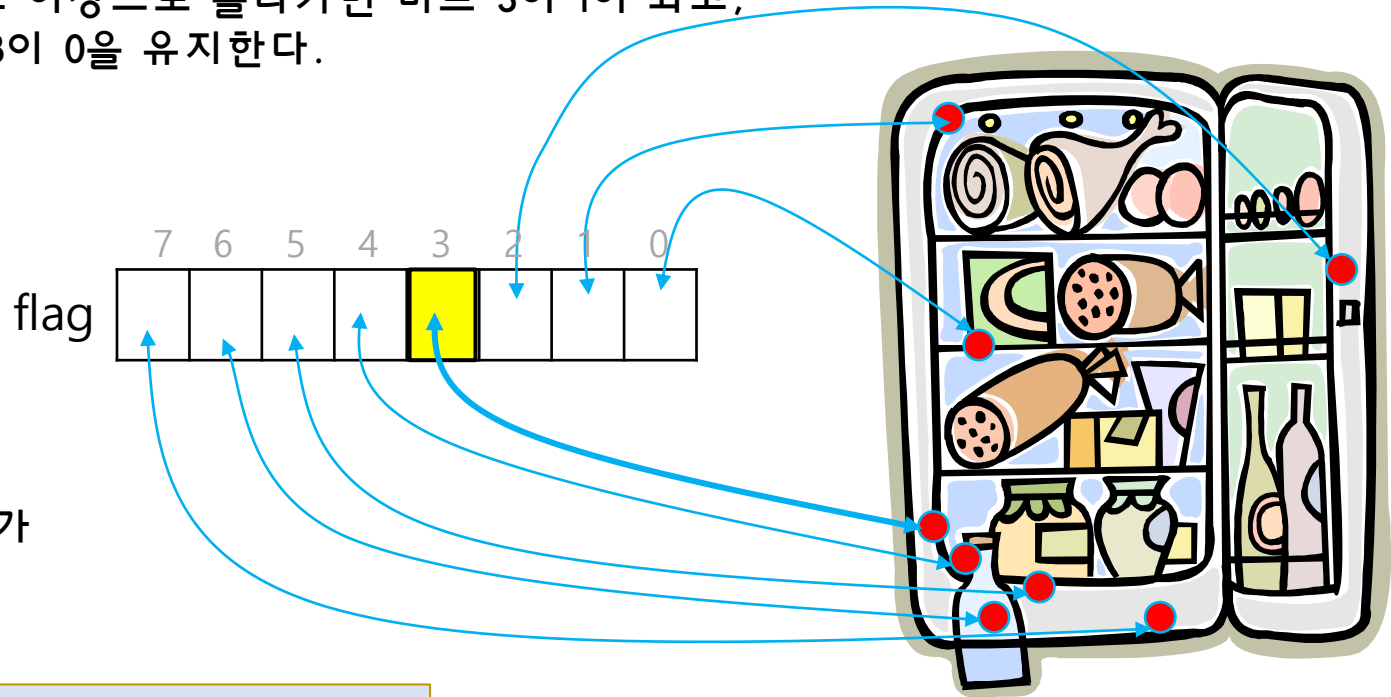


```
byte a = (byte)0xf8; // -8
byte b = (byte)(a >> 2); // -2
```



# 예제. 비트 연산 응용

냉장고에는 8개의 센서가 있고 이들은 flag 변수와 연결되어 있다고 할 때,  
냉장고의 온도가 0도 이상으로 올라가면 비트 3이 1이 되고,  
0도 이하이면 비트 3이 0을 유지한다.



문제)  
현재 냉장고의 온도가  
0도 이상인지  
판단하는 코드는?

```
byte flag = 0b00001010;
// 각 비트는 8개의 센서 값을 가리킴
if(flag & 0b00001000 == 0)
 System.out.print("온도는 0도 이하");
else
 System.out.print("온도는 0도 이상");
```

온도는 0도 이상

|       |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|
|       | x | x | x | x | Y | x | x | x |
| &     | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| <hr/> |   |   |   |   |   |   |   |   |
|       | 0 | 0 | 0 | 0 | Y | 0 | 0 | 0 |

Y비트가 0이면  
& 결과는 0

# 논리 연산자와 단락 평가

- 논리 연산자
  - && (AND), || (OR), ! (NOT) — boolean 값을 대상으로 연산
- 단락 평가 (short-circuit evaluation)
  - 앞의 식만으로 결과가 확정되면 뒤의 식은 평가하지 않음
    - false && 식 → 뒤를 평가하지 않음, true || 식 → 뒤를 평가하지 않음
  - &, | 를 boolean에 사용하면 항상 양쪽 모두 평가함
    - 양쪽 피연산자가 모두 boolean이면 &는 비트 연산이 아니라 단락평가를 하지 않는 논리 AND / OR로 동작함

```
int n = 0;

// n이 0이면 뒤의 나눗셈을 평가하지 않으므로 안전함
if (n != 0 && 100 / n > 10)
 System.out.println("몫이 10보다 큼니다.");

// &를 사용하면 100/0이 실행되어 ArithmeticException 발생!
// if (n != 0 & 100 / n > 10)
```

# 조건문

\* if문은 C언어와 동일함

# switch문

- switch문은 식과 case 문의 값과 비교
  - case의 비교 값과 일치하면 해당 case의 실행문장 수행
    - break를 만나면 switch문을 벗어남
  - case의 비교 값과 일치하는 것이 없으면 default 문 실행
    - default문은 생략 가능

```
switch(식) {
 case 값1: // 식의 결과가 값1과 같을 때
 실행 문장 1;
 break;
 case 값2: // 식의 결과가 값2와 같을 때
 실행 문장 2;
 break;
 ...
 case 값m:
 실행 문장 m; // 식의 결과가 값m과 같을 때
 break;
 default: // 어느 것과도 같지 않을 때
 실행 문장 n;
}
```

```
char grade='B';
switch(grade) {
 case 'A':
 System.out.println("축하합니다.");
 System.out.println("잘했습니다.");
 break;
 case 'B':
 System.out.println("좋아요.");
 break;
 case 'C':
 System.out.println("노력하세요.");
 break;
 default:
 System.out.println("탈락입니다!");
}
```

# switch문에서 break문의 역할

- switch문 내의 break문
  - break문을 만나면 switch문 벗어남
  - case 문에 break문이 없다면, 다음 case문으로 실행 계속
    - 언젠가 break를 만날 때까지 계속 내려 가면서 실행

break; 문을  
생략한다면

```
char grade='A';
switch (grade) {
 case 'A':
 System.out.println("90 ~ 100점입니다.");
 break;
 case 'B':
 System.out.println("80 ~ 89점입니다.");
 break;
 case 'C':
 System.out.println("70 ~ 79점입니다.");
 break;
}
```

90 ~ 100점입니다.  
80 ~ 89점입니다.

# case 문의 값

- case 문의 값
  - 문자, 정수, **문자열 리터럴**(JDK 1.7부터) 만 허용
  - 실수 리터럴은 허용되지 않음

```
int b;
switch(b%2) {
 case 1 : ...; break;
 case 2 : ...; break;
}
```

정수 리터럴  
사용 가능

```
char c;
switch(c) {
 case '+' : ...; break;
 case '-' : ...; break;
}
```

문자 리터럴  
사용 가능

```
String s = "예";
switch(s) {
 case "예" : ...; break;
 case "아니요" : ...; break;
}
```

문자열 리터럴  
사용 가능

```
switch(a) {
 case a : // 오류. 변수 사용 안됨
 case a > 3 : // 오류. 수식 안됨
 case a == 1 : // 오류. 수식 안됨
}
```

오류

# 예제. switch 문 사용하기

- 문제 내용
  - 1~12 사이의 월을 입력 받아 봄, 여름, 가을, 겨울을 판단하여 출력하라.
  - switch 표현식을 이용하여 해결하자.

월(1~12)을 입력하시오:3  
봄입니다.

# 예제. switch 문 사용하기

```
public class Season {
 public static void main(String[] args) {
 Scanner scanner = new Scanner(System.in);
 System.out.print("월(1~12)을 입력하시오:");
 int month = scanner.nextInt(); // 정수로 월 입력
 switch(month) {
 case 3:
 case 4:
 case 5:
 System.out.println("봄입니다.");
 break;
 case 6: case 7: case 8:
 System.out.println("여름입니다.");
 break;
 case 9: case 10: case 11:
 System.out.println("가을입니다.");
 break;
 case 12: case 1: case 2:
 System.out.println("겨울입니다."); break;
 default:
 System.out.println("잘못된 입력입니다.");
 }
 }
}
```

# Switch 표현식 확장

- Jdk14 부터 확장된 표현식 제공
  - 불필요한 반복코드 존재 (case, break)
  - break 구문 주의 사용해야 함

```
switch (day) {
 case MONDAY:
 case FRIDAY:
 case SUNDAY:
 System.out.println(6);
 break;
 case TUESDAY:
 System.out.println(7);
 break;
 case THURSDAY:
 case SATURDAY:
 System.out.println(8);
 break;
 case WEDNESDAY:
 System.out.println(9);
 break;
}
```

```
switch (day) {
 case MONDAY, FRIDAY, SUNDAY -> System.out.println(6);
 case TUESDAY -> System.out.println(7);
 case THURSDAY, SATURDAY -> System.out.println(8);
 case WEDNESDAY -> System.out.println(9);
}
```

- case에 대한 multiple constants 가능
- -> 다음에는 표현식, { }, throw 구문으로 제한되며, break 구문 필요 없음

\*참고 : <https://openjdk.org/jeps/361>

# Switch 표현식 확장

- 변수 대입도 가능함

```
int numLetters;
switch (day) {
 case MONDAY:
 case FRIDAY:
 case SUNDAY:
 numLetters = 6;
 break;
 case TUESDAY:
 numLetters = 7;
 break;
 case THURSDAY:
 case SATURDAY:
 numLetters = 8;
 break;
 case WEDNESDAY:
 numLetters = 9;
 break;
 default:
 throw new IllegalStateException("Wat: " + day);
}
```

```
int numLetters = switch (day) {
 case MONDAY, FRIDAY, SUNDAY -> 6;
 case TUESDAY -> 7;
 case THURSDAY, SATURDAY -> 8;
 case WEDNESDAY -> 9;
};
```

```
static void howMany(int k) {
 switch (k) {
 case 1 -> System.out.println("one");
 case 2 -> System.out.println("two");
 default -> System.out.println("many");
 }
}
```

```
static void howMany(int k) {
 System.out.println(
 switch (k) {
 case 1 -> "one";
 case 2 -> "two";
 default -> "many";
 }
);
}
```

# 실습 예제. 계절 출력하기

- 문제 내용
  - 1~12 사이의 월을 입력 받아 봄, 여름, 가을, 겨울을 판단하여 출력하라
  - 앞에서 살펴본 예제를 확장된 switch 표현식으로 바꾸기

# **반복문**

**for 문, while 문, do-while문**

\* 기본 문법 C언어와 동일함

# for 문

- for문의 예
  - 반복문에 변수 선언 가능
    - for문을 벗어나면 사용할 수 없음

```
for(int i=0; i<10; i++)
 System.out.print(i);
```

```
int i, sum;
for(i=0; i<100; i++){
 sum += i;
 if(sum>100) break;
}
System.out.print(i);
```

- 무한반복 사용 가능

```
for(초기문; true; 반복 후 작업) {

}
```

```
for(초기문; ; 반복 후 작업) {

}
```

# while 문

- while 문의 구성과 코드 사례

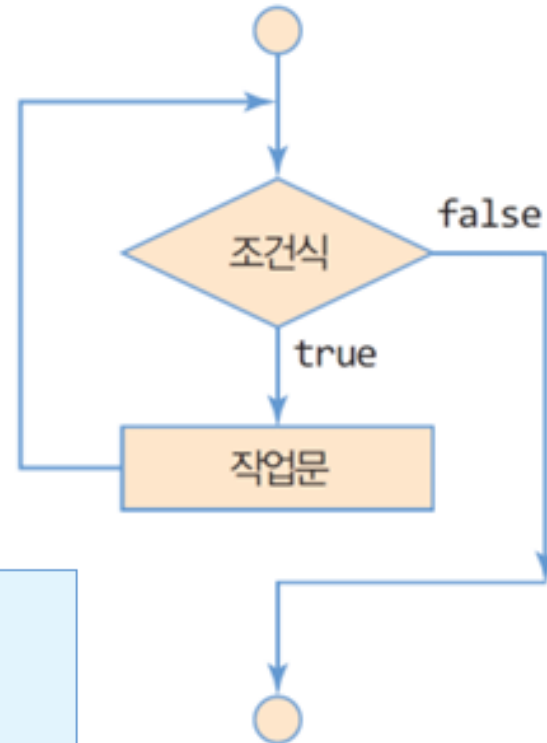
`while(조건식) {`  
    `.. 작업문 ..`  
`}`

1 (조건식)  
2 (작업문)

- 조건식이 '참'인 동안 반복 실행

```
int i=0;
while(i<10) { // 0에서 9까지 출력
 System.out.print(i);
 i++;
}
```

0123456789



# do-while 문

- do-while 문의 구성과 코드 사례
  - 조건식이 '참'인 동안 반복 실행
  - 작업문은 한 번 반드시 실행

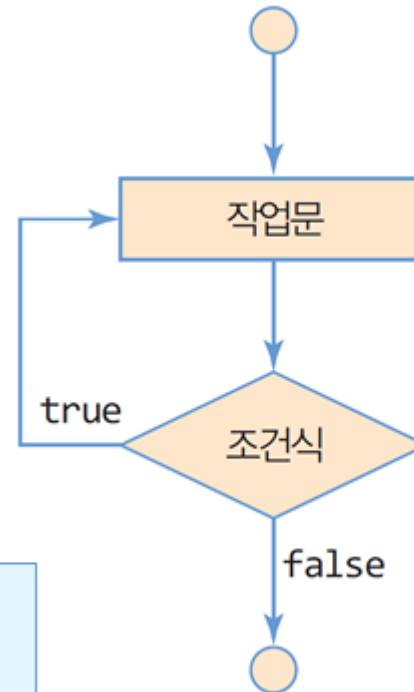
```
do {
 .. 작업문 ..
} while(조건식);
```

1

2

```
int i=0;
do { // 0에서 9까지 출력
 System.out.print(i);
 i++;
} while(i<10);
```

0123456789



# continue문

- continue 문
  - 반복문을 빠져 나가지 않고, 다음 반복으로 제어 변경
  - 반복문에서 continue; 문에 의한 분기

```
for(초기문; 조건식; 반복 후 작업) {

 continue;

}
```

분기

```
while(조건식) {

 continue;

}
```

조건식으로 분기

```
do {

 continue;

} while(조건식);
```

조건식으로 분기

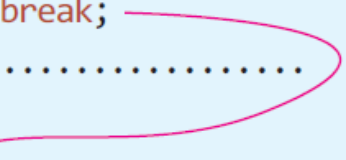
# break문

- break 문
  - 반복문 하나를 즉시 벗어갈 때 사용
    - 하나의 반복문만 벗어남
    - 중첩 반복의 경우 안쪽 반복문의 break 문이 실행되면 안쪽 반복문만 벗어남

```
for(초기문; 조건식; 반복 후 작업) {

 break;

}
.....
```



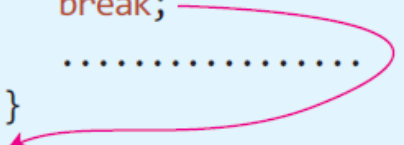
(a) 하나의 반복문을 벗어나는 경우

```
for(초기문; 조건식; 반복 후 작업) {
 while(조건식) {

 break;

 }

}
.....
```



(b) 중첩 반복에서 안쪽 반복문만 벗어나는 경우

# 중첩 반복문과 구구단

- 중첩 반복문 (nested loop)
  - 반복문 안에 또 다른 반복문을 포함하는 구조
  - 바깥 반복이 1회 실행될 때마다 안쪽 반복은 처음부터 끝까지 실행
  - 구구단 : 바깥 반복(2~9단) × 안쪽 반복(각 단의 1~9)

## <코드>

```
for (int dan = 2; dan <= 9; dan++) {
 for (int i = 1; i <= 9; i++) {
 System.out.printf("%d x %d = %2d%n", dan, i, dan * i);
 }
 System.out.println(); // 단 구분 빈 줄
}
```

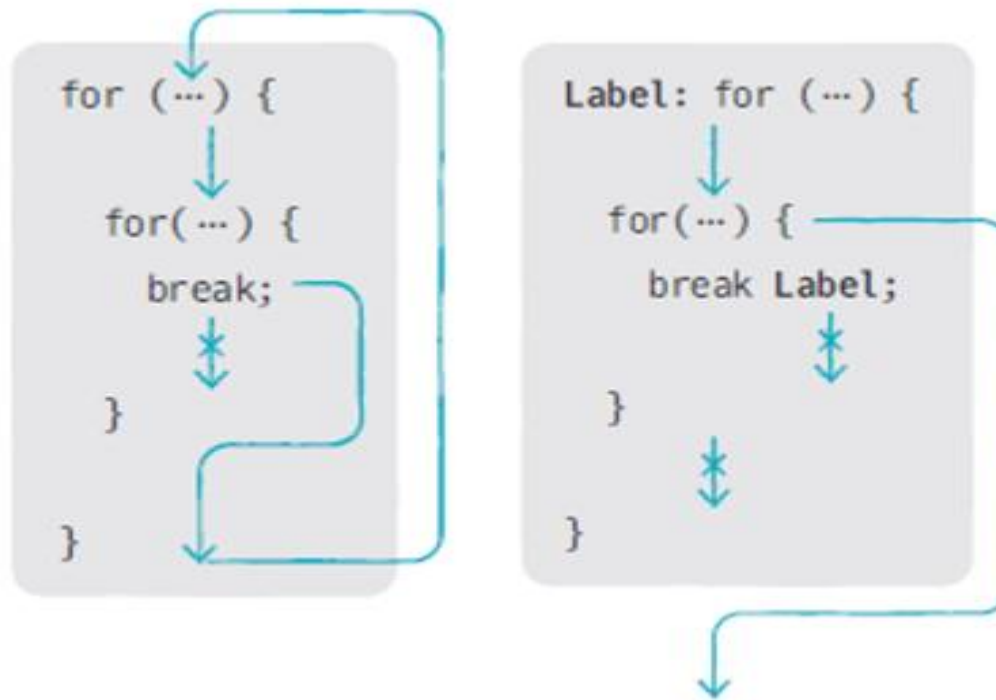
## <출력 예시>

```
2 x 1 = 2
2 x 2 = 4
...
2 x 9 = 18

3 x 1 = 3
...
9 x 9 = 81
```

# break label문

- 반복문이 중첩된 경우 원하는 label 위치의 반복문을 빠져나가기 위해 사용하는 구문



# 실습 예제. 별 찍기

- 문제 설명
  - 정수  $n$ 을 입력 받아  $n$ 줄의 직각삼각형 모양으로 별(\*)을 출력하는 프로그램을 작성하라. (중첩 for문 사용)
- 코딩 전 생각할 것
  - 바깥 반복 : 줄 번호  $i$  ( $1 \sim n$ )
  - 안쪽 반복 : 각 줄에 출력할 별의 개수 ( $1 \sim i$ )

<출력 예시>

```
줄 수를 입력하시오:5
*
**


```

도전) 별을 오른쪽 정렬로 출력하세요.

# 실습 예제. 마름모 찍기 (도전!!!)

- 문제 설명
  - 홀수  $n$ 을 입력 받아 높이  $n$ 의 마름모 모양으로 별(\*)을 출력하는 프로그램을 작성하라.
- 코딩 전 생각할 것
  - 위쪽 삼각형(1~가운데 줄)과 아래쪽 삼각형으로 나누어 생각
  - 각 줄 = 공백 반복 + 별 반복  $\rightarrow$  중첩 반복문 2개 조합
  - 줄 번호  $i$ 에 따른 공백 개수와 별 개수의 규칙을 먼저 표로 정리해 보자

## <출력 예시>

높이(홀수)를 입력하시오:5

```
*

*
```

힌트)  $i$ 번째 줄의 별 개수 :  $2*i-1$  (위쪽 기준)  
공백 개수 :  $(n/2+1) - i$

수고하셨습니다.